Ders 9 Çalışma Özeti

Ders 9 Çalışma Özeti

Genel Konular

- Transport Katmanına Giriş
  - Network katmanı güvenilir değildir; paketler kaybolabilir, bozulabilir, sırasız gelebilir, duplicate olabilir. Transport katmanı bu güvensizliği üst katmana "güvenilir" olarak yansıtır.
  - Servisler: connectionless (UDP) veya connection-oriented (TCP) — datagram veya virtual circuit.
  - Network'te dört kombinasyon mümkündür; network'te hangisinin seçildiği önemlidir.
- Transport Katmanının Temel Görevleri
  - Üst katmana güvenilirlik sağlamak (hata kontrolü, sıralama, tekrar gönderim).
  - Uygulamalar arası multiplexing/demultiplexing (port numaraları).
  - Akış kontrolü (flow control).
  - Tıkanıklık kontrolü (congestion control).
- Transport Adresleme ve Port
  - Network adresi (IP) host'u tanımlar; transport adresi (port) o host'taki uygulamayı/prosesi tanımlar.
  - Port 16-bit (0-65535); 0-1023 "well-known port" (örn. 80=HTTP, 443=HTTPS, 22=SSH, 25=SMTP, 53=DNS, 110=POP3, 143=IMAP).
  - Apartman/daire benzetmesi: aynı IP altında farklı daireler (portlar) farklı kiracılara (prosesler) kiralanır.
  - TCP/IP'de transport adres = port numarası; OSI terminolojisinde "TSAP" (Transport Service Access Point).
- Segment, Paket, Frame, Message
  - **Message**: Uygulama katmanında değiş tokuş edilen veri bütünü; teorik olarak sınırsız büyüklükte.
  - **Segment**: Transport katmanı PDU'su (Protocol Data Unit); header + payload.
  - **Packet / Datagram**: Network katmanı PDU'su.
  - **Frame**: Data link katmanı PDU'su.
  - Rus matruşka bebek gibi iç içe geçmiş yapı.
- Primitifler (Connection-Oriented Servis)
  - **Listen**: Server tarafında dinleme pozisyonuna geçme.
  - **Connect**: Client'ın bağlantı başlatma isteği (connection request).
  - **Send**: Veri gönderme.
  - **Receive**: Veri alma.
  - **Disconnect**: Bağlantı sonlandırma.
  - **Accept**: Server tarafında gelen isteği kabul etme (multiplexing için kritik).
- Durum Diyagramı (State Diagram)
  - Connection-oriented protokol için durumlar ve geçişler tanımlanmalıdır.
  - **Idle**: Pasif bekleme.
  - **Passive Establishment Pending**: Server bağlantı bekliyor.
  - **Active Establishment Pending**: Client bağlantı başlatmış.
  - **Established**: Bağlantı kurulmuş, veri alışverişi.
  - **Passive/Active Disconnect Pending**: Bağlantı sonlandırma aşamasında.
  - **Idle**: Bağlantı sona ermiş.
  - Network layer "vahşi batı" gibi çalışır (kayıp, sırasız, bozuk); transport layer bu kaosu

düzene sokar.
- Socket, Bind, Listen, Accept
  - **Socket**: Transport adresini tanımlayan veri yapısı (file descriptor gibi).
  - **Bind**: Soket'i belirli bir IP ve port'a bağlama.
  - **Listen**: Dinleme kuyruğu oluşturma (backlog parametresi ile).
  - **Accept**: Gelen bağlantı isteğini kabul etme; multiplexing tablosu kullanarak her isteği farklı iç porta yönlendirme.
  - Örnek: Google 80 portunu dinler; gelen istekler 10000-10740 arası iç portlara dağıtılır. Telefon santralı benzetmesi.
- Connection Establishment Problemleri
  - Gecikmiş (delayed) paket: Karşı taraf cevap gelmeyince zaman aşımı ile yeni istek gönderir; eski cevap geç gelirse, karşı taraf aynı bağlantıyı iki kez kurmuş olur.
  - Duplicate paket: Aynı paket iki kez gelirse, eski durum ile yeni durum karıştırılabilir.
  - Çözüm: 3-way handshake (SYN, SYN+ACK, ACK) ve sequence number kullanımı.
  - Gecikmenin üst sınırı bilinmediğinden, üst sınır koyup aşan paketleri "kayıp" saymak gerekir.

Hocanın Özellikle Vurguladığı Kısımlar

- Connectionless vs connection-oriented dört kombinasyon
  - Network layer connectionless + transport connectionless → UDP.
  - Network layer connectionless + transport connection-oriented → TCP.
  - Diğer iki kombinasyon teorik olarak mümkün ama pratikte yaygın değil.
- Port numarasının önemi
  - Bir IP adresi tek başına yeterli değildir; aynı IP'de birden fazla uygulama çalışabilir (örn. web + mail + ssh). Port, bu ayrımı sağlar.
- Accept'in multiplexing'deki rolü
  - Eğer sadece bir bağlantı kabul edilebilse, Google aynı anda milyonlarca kullanıcıya hizmet veremezdi. Accept, gelen istekleri farklı iç portlara dağıtarak multiplexing sağlar.
- Network layer'ın "vahşi batı" oluşu
  - Hoca, network layer için "kelimenin tam anlamıyla vahşi batı" der; paketler kaybolur, sırasız gelir, her türlü sorun yaşanır. Transport layer, bu kaotik ortamı düzene sokar.

Kısa Tekrar Notları

- Transport = network üstünde güvenilirlik
- Port = transport adresi (16 bit)
- 0-1023 well-known portlar
- Segment > packet > frame (iç içe)
- Primitifler: Listen, Connect, Send, Receive, Disconnect, Accept
- Multiplexing: Accept ile farklı iç portlara dağıtım
- 3-way handshake: SYN, SYN+ACK, ACK

Detaylı Açıklamalar

Transport katmanı, network katmanının güvensizliğini soyutlar. Network katmanı "best-effort" çalışır; en iyi çabayı gösterir ama garanti vermez. Transport katmanı ise uygulamaya "sıralı, hatasız, kayıpsız veri akışı" sözü verir. Bu, tekrar gönderim, sıralama, hata tespiti gibi mekanizmalarla sağlanır.

Port kavramı, transport katmanının en temel bileşenidir. Aynı IP adresine sahip bir sunucu, farklı portlarda farklı hizmetler sunabilir. Örneğin, bir sunucu 80 portunda HTTP, 22 portunda SSH, 25 portunda SMTP hizmeti verebilir. Bu sayede tek bir IP, birden fazla rol üstlenir.

Soket programlama, port kavramının pratiğe dökülmüş halidir. C'deki `socket()`, `bind()`, `listen()`, `accept()` fonksiyonları, transport adreslerini oluşturur, bağlar, dinler ve kabul eder. Aynen bir dosya açmak gibi, ama dosya yerine network bağlantısı. Bu soyutlama, programcıların dosya işlemleri gibi network işlemleri yapmasını sağlar.

Connection establishment, güvenilir bir bağlantının kurulması sürecidir. En temel sorun: network'te duplicate veya gecikmiş paketler olabilir. Eğer bir bağlantı isteği kaybolursa, gönderici tekrar gönderir. Karşı taraf bunu yeni bir istek olarak algılar ve bağlantı kurar. Ancak aslında bu, aynı bağlantının iki kez kurulması demektir. Çözüm: sequence number kullanmak. Her bağlantı isteğine benzersiz bir numara verilir; karşı taraf bunu hatırlar ve aynı numarayla gelen tekrarı reddeder. 3-way handshake (SYN → SYN+ACK → ACK) bu mantıkla çalışır.

Hoca, gecikmiş paketler için pratik bir sınır koyma gerekliliğini vurgular. Network'te bir paketin en fazla ne kadar gecikebileceğini bilemeyiz; bu nedenle bir üst sınır (timeout) belirlenir. Bu süreyi aşan paket "kayıp" sayılır ve yeniden gönderilir. Bu, basit ama etkili bir yaklaşımdır; gerçek dünyada da benzer şekilde çalışır (örn. bir mektup uzun süre gelmezse, karşı taraf "gelmemiş" sayar).

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.